

REGULARIZATION

Lasso, Ridge Regression, and Elastic Nets

Marco R. Steenbergen 

steenbergen@ipz.uzh.ch

University of Zurich

July 20, 2026

R Packages for this Lecture

Next to the packages from the last time, we need:

- **future** and **doFuture** for parallel processing during tuning (speeds things up).
- **glmnet** for regularized linear and logistic regression.

What Is Regularization?

The Basic Idea

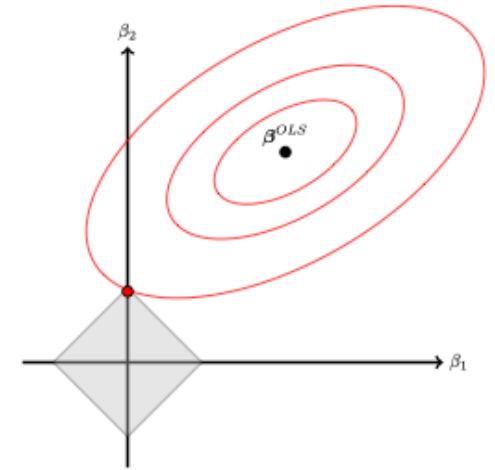
- **Regularization** adds a penalty for over-fitting to the loss function.
- This penalty serves as a constraint on the optimization problem.
- We can also think of this as a shrinkage estimator, with small coefficients being shrunk more than larger coefficients.
- The most important regularization procedures are
 1. The lasso.
 2. Tikhonov regularization/ridge regression.
 3. Elastic nets.

The Lasso

- **Lasso** = least absolute shrinkage and selection operator ([Tibshirani 1996](#)).
- We impose $\sum_{j=1}^P |\beta_j| \leq t$.
- Using Lagrange multipliers, the lasso loss is $L_{\text{lasso}} = L_2 + \lambda \left(\sum_{j=1}^P |\beta_j| - t \right)$.
- Retaining only the terms in ω , the optimization problem is

$$\min_{\alpha, \beta} \sum_{i=1}^{n_1} \left(\alpha + \mathbf{x}_i^\top \boldsymbol{\beta} - y_i \right)^2 + \lambda |\boldsymbol{\beta}|$$

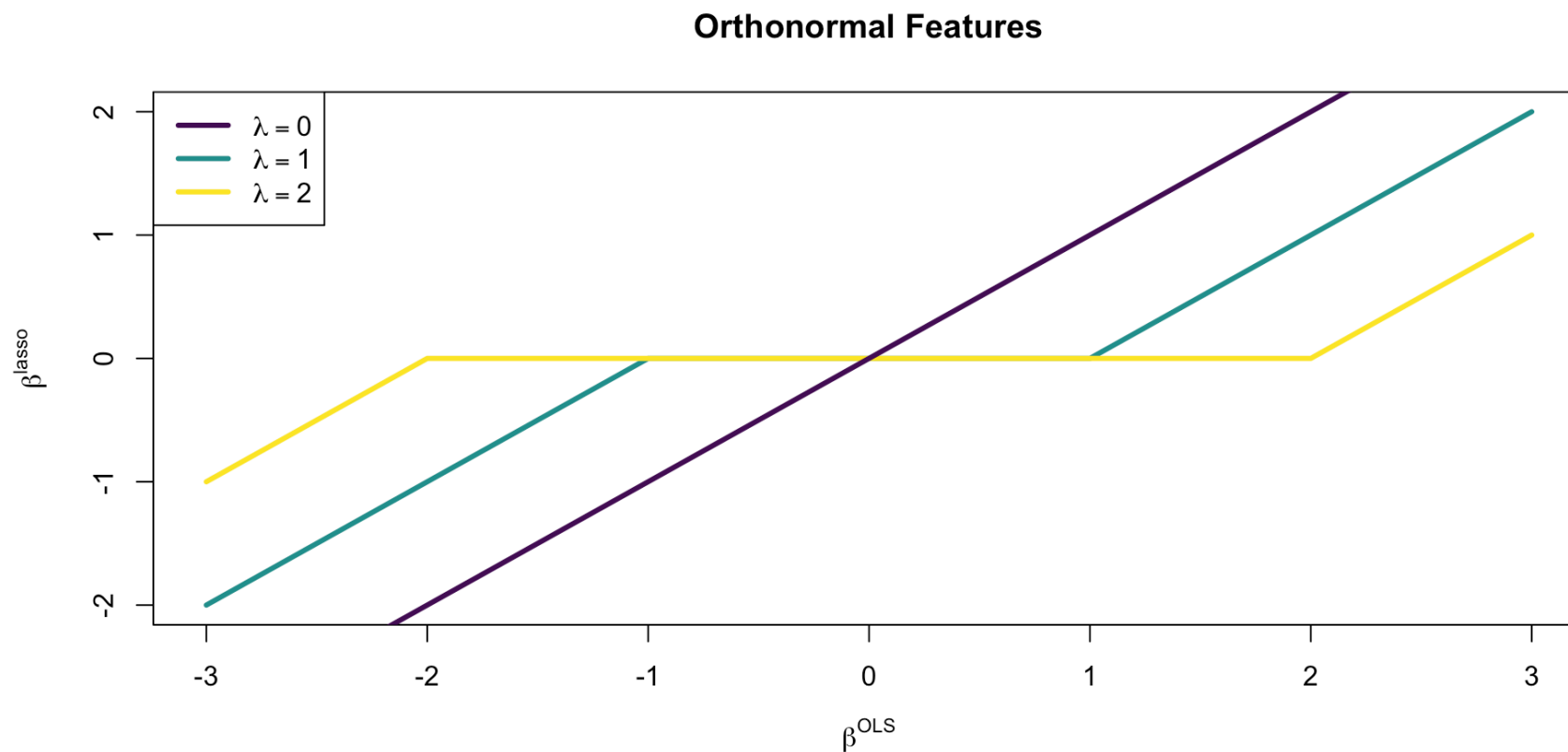
- λ is a hyper-parameter.



What the Lasso Does

1st Example

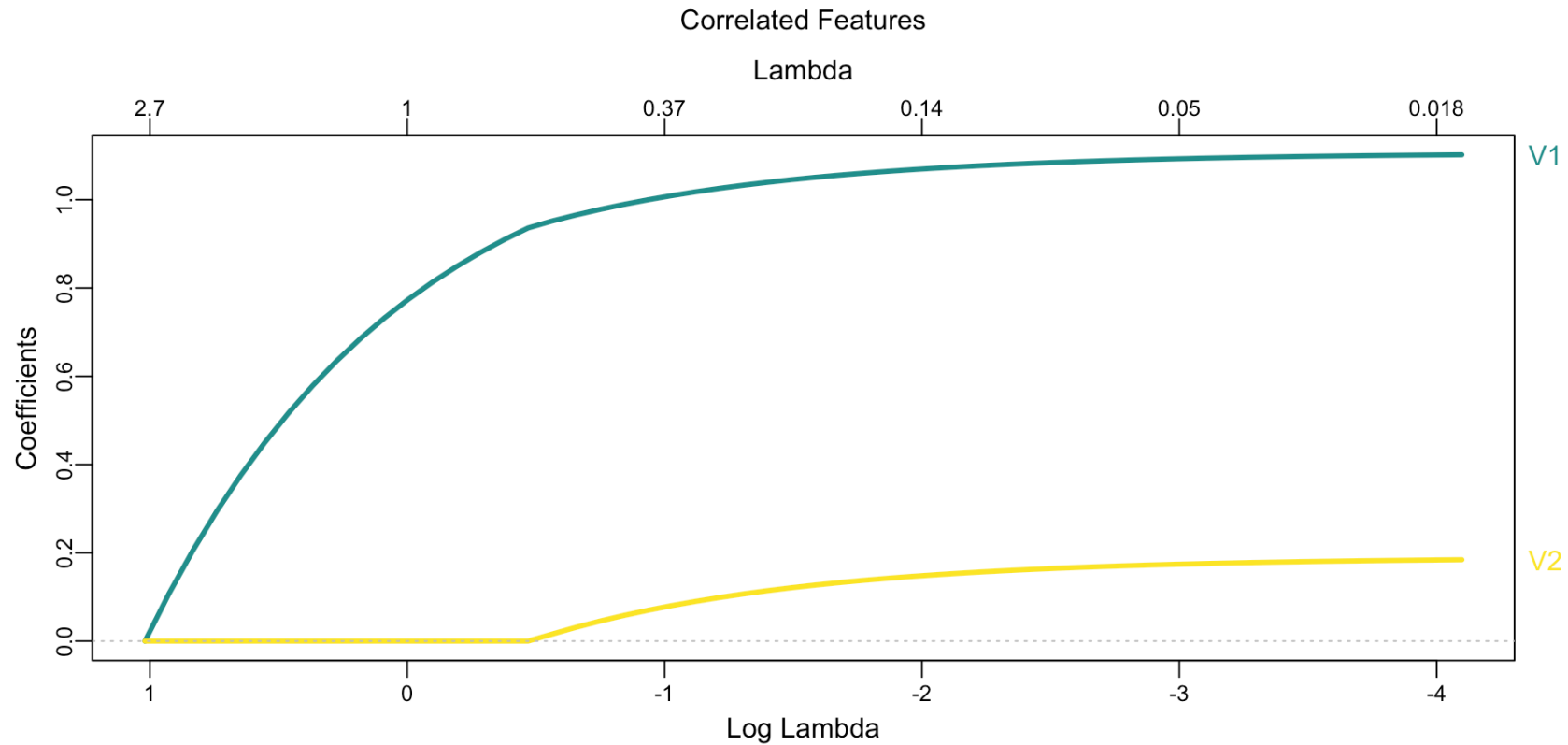
2nd Example



What the Lasso Does

1st Example

2nd Example



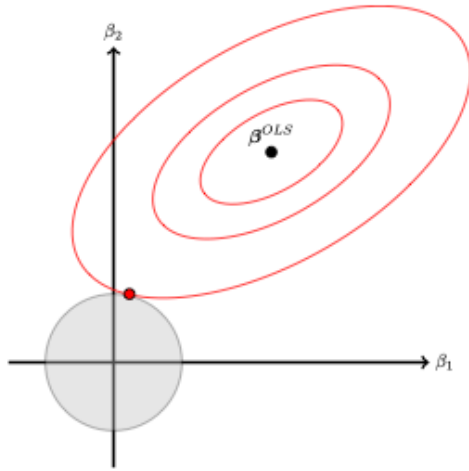
Variations on a Theme

- We can think of the lasso as a special case of

$$\min_{\alpha, \boldsymbol{\beta}} \sum_{i=1}^{n_1} \left(\alpha + \mathbf{x}_i^\top \boldsymbol{\beta} - y_i \right)^2 + \lambda |\boldsymbol{\beta}|^q$$

- For $q = 1$, this becomes the lasso.
- For $q = 2$, this becomes **Tikhonov regularization** (a.k.a. ridge regression).

Tikhonov Regularization



- We minimize the L_2 -norm subject to $\sum_{j=1}^P \beta_j^2 \leq t$ (Tikhonov 1963).
- Hence,

$$\min_{\alpha, \beta} \sum_{i=1}^{n_1} (\alpha + \mathbf{x}_i^\top \boldsymbol{\beta} - y_i)^2 + \lambda \sum_{j=1}^P \beta_j^2$$

Elastic Nets

- Elastic nets are a mixture of the lasso and Tikhonov regularization ([Zou and Hastie 2005](#)).
- For $\gamma \in [0, 1]$, define the penalty

$$P(\gamma) = \sum_{j=1}^P \left[\frac{1}{2} (1 - \gamma) \omega_j^2 + \gamma |\omega_j| \right]$$

- Then

$$\min_{\omega} \sum_{i=1}^{n_1} \left(\beta + \mathbf{x}_i^{\top} \omega - y_i \right)^2 + \lambda P(\gamma)$$

Elastic Nets Cont'd

- Note this reduces to
 - the lasso for $\gamma = 1$.
 - Tikhonov regularization for $\gamma = 0$.
- Here γ is another hyper-parameters.

Important

- A valid use of regularization requires that features are put on the same scale.
- `tidymodels` offers several ways to accomplish this:
 - `step_normalize()` standardizes the features.
 - `step_range()` scales the minimum and maximum to 0 and 1, respectively.

How to Select and Validate Hyper-Parameters

A Wealth of Possibilities

- In rare cases—e.g., principal component analysis—hyper-parameters take on a finite number of discrete values $\Rightarrow$ grid search: try each value.
- This is not our case: γ and λ are both continuous and there are endless combinations.
- The simplest possibility is to do a random search:¹
 - $\rightarrow$ Max entropy when there is 1 hyper-parameter.
 - $\rightarrow$ Latin hyper-cubes with max entropy when there are multiple hyper-parameters.

1. SMBO will be introduced later in the course.

Space-Filling and Latin Hyper-cubes

- **Goal:** For a K -dimensional parameter space, we must ensure uniform coverage across all dimensions.
 - Standard random sampling leaves empty voids and redundant clusters.
 - Space-filling designs (like Latin Hyper-cubes) force points to spread evenly ([Husslage et al. 2011](#); [McKay et al. 1979](#)).
- **Maximum Entropy Designs** (e.g., Kriging / Gaussian Process approach):
 - Model hyperparameter locations as a spatial process where points are correlated.
 - The **variogram_range** parameter controls the spatial decay / effective repulsion scale.
 - **Higher variogram_range values** expand the repulsion field, reducing the likelihood of large empty voids in the space.

Evaluating the Grid

- Now that we have set the grid, how do we evaluate different choices of hyper-parameter values?
- The basic idea is this:
 1. Set (γ, λ) .
 2. Train the model with those values.
 3. Evaluate performance without touching the test data—they come at the very end.
- The problem: resubstitution error ([Efron 1983](#)).
- We need to set aside data within the training set to **validate** our choice of hyper-parameters.

Building Validation Sets

1. The validation split.
2. v -Fold cross-validation.
3. Bootstrapping.

The Validation Split

- Take the n_1 training instances.
- Set a probability of q for training use and $1 - q$ for validation use:
 - $q \cdot n_1$ training instances proper—used to estimate $\theta|\tau$, where τ is a set of hyper-parameter values from the grid.
 - $(1 - q) \cdot n_1$ validation instances—used to assess predictive performance given τ .
- We iterate over the grid and pick as τ that set of values that maximize performance in the **validation set**.

The Validation Split Cont'd

Data

Validation

```
1 library(mlbench)
2 library(tidymodels)
3 library(tidyverse)
4 tidymodels_prefer()
5 data("BostonHousing2")
6 df_new <- BostonHousing2 |>
7   mutate(target = log10(cmedv),
8           high_tax = ifelse(tax > 600, 1, 0)) |>
9   select(tract, target, nox, tax, high_tax)
10 set.seed(1292)
11 housing_split <- initial_split(df_new, prop = 0.75)
12 train_df <- training(housing_split)
```

The Validation Split Cont'd

Data

Validation

```
1 # We set q = 0.75
2 set.seed(2158)
3 validation_set <- validation_split(train_df, prop = 3/4)
4 validation_set
```

```
# Validation Set Split (0.75/0.25)
```

```
# A tibble: 1 × 2
```

```
  splits          id
  <list>        <chr>
```

```
1 <split [284/95]> validation
```

The Validation Split Cont'd

- Advantage: Only have to train once per combination of hyper-parameters.
- Disadvantages:
 1. We lose training data: $q \cdot n_1 < n_1$.
 2. We may end up with an atypical validation set.
 3. We obtain only one performance value per combination.
- In large samples, the first two problems lose relevance and the computational advantage often outweighs the third disadvantage.
- In medium-sized data, the disadvantages loom larger ([Molinaro et al. 2005](#)).
- But can we do better?

v -Fold Cross-Validation

TRAIN	TRAIN	VALIDATE
TRAIN	VALIDATE	TRAIN
VALIDATE	TRAIN	TRAIN

v -Fold Cross-Validation steps:

- Randomly assign instances to k folds (typically, $v = 10$).
- Each fold is held out once for validation purposes $\Rightarrow v$ performance estimates per τ .
- Each fold is used $v - 1$ times for training purposes $\Rightarrow$ no loss of training data.

Variations on a Theme

- The higher we set k , the more data will be used for training at any given time.
- An extreme case is **leave out one CV** (LOOCV), where $k = n_1 - 1$.
- In each run, bias is reduced but variance is increased.
- Another way to reduce bias is to **repeatedly** assign instance to the folds, each time using a different seed.
- **Monte Carlo CV** selects the folds randomly each time, which may cause folds to contain overlapping instances ([Xu and Liang 2001](#)).

v -Fold Cross-Validation in `tidymodels`

Vanilla

Repeated

LOOCV

Monte Carlo

```
1 set.seed(1923)
2 vanilla_folds <- vfold_cv(train_df, v = 10)
3 vanilla_folds
```

10-fold cross-validation

A tibble: 10 × 2

	splits	id
	<list>	<chr>
1	<split [341/38]>	Fold01
2	<split [341/38]>	Fold02
3	<split [341/38]>	Fold03
4	<split [341/38]>	Fold04
5	<split [341/38]>	Fold05
6	<split [341/38]>	Fold06
7	<split [341/38]>	Fold07
8	<split [341/38]>	Fold08
9	<split [341/38]>	Fold09
10	<split [342/37]>	Fold10

v -Fold Cross-Validation in `tidymodels`

Vanilla

Repeated

LOOCV

Monte Carlo

```
1 set.seed(1923)
2 repeated_folds <- vfold_cv(train_df, v = 10, repeats = 5)
3 repeated_folds
```

```
# 10-fold cross-validation repeated 5 times
```

```
# A tibble: 50 × 3
```

	splits	id	id2
	<list>	<chr>	<chr>
1	<split [341/38]>	Repeat1	Fold01
2	<split [341/38]>	Repeat1	Fold02
3	<split [341/38]>	Repeat1	Fold03
4	<split [341/38]>	Repeat1	Fold04
5	<split [341/38]>	Repeat1	Fold05
6	<split [341/38]>	Repeat1	Fold06
7	<split [341/38]>	Repeat1	Fold07
8	<split [341/38]>	Repeat1	Fold08
9	<split [341/38]>	Repeat1	Fold09
10	<split [342/37]>	Repeat1	Fold10

```
# i 40 more rows
```

v -Fold Cross-Validation in `tidymodels`

Vanilla

Repeated

LOOCV

Monte Carlo

```
1 set.seed(1923)
2 loocv_folds <- loo_cv(train_df)
3 loocv_folds
```

Leave-one-out cross-validation

A tibble: 379 × 2

	splits	id
	<list>	<chr>
1	<split [378/1]>	Resample1
2	<split [378/1]>	Resample2
3	<split [378/1]>	Resample3
4	<split [378/1]>	Resample4
5	<split [378/1]>	Resample5
6	<split [378/1]>	Resample6
7	<split [378/1]>	Resample7
8	<split [378/1]>	Resample8
9	<split [378/1]>	Resample9
10	<split [378/1]>	Resample10

i 369 more rows

v -Fold Cross-Validation in `tidymodels`

Vanilla

Repeated

LOOCV

Monte Carlo

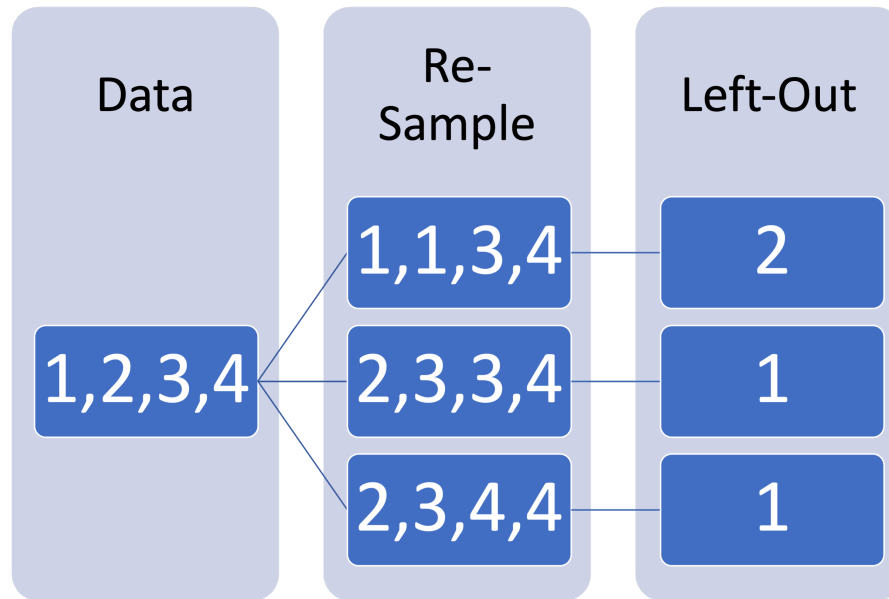
```
1 set.seed(1923)
2 mc_folds <- mc_cv(train_df, prop = 9/10, times = 20)
3 mc_folds
```

Monte Carlo cross-validation (0.9/0.1) with 20 resamples

A tibble: 20 × 2

	splits	id
	<list>	<chr>
1	<split [341/38]>	Resample01
2	<split [341/38]>	Resample02
3	<split [341/38]>	Resample03
4	<split [341/38]>	Resample04
5	<split [341/38]>	Resample05
6	<split [341/38]>	Resample06
7	<split [341/38]>	Resample07
8	<split [341/38]>	Resample08
9	<split [341/38]>	Resample09
10	<split [341/38]>	Resample10
11	<split [341/38]>	Resample11
12	<split [341/38]>	Resample12
13	<split [341/38]>	Resample13
14	<split [341/38]>	Resample14

Bootstrapping



- The **bootstrap** consists of repeatedly sampling n_1 instances with replacement ([Efron 1979](#)).
- Sampled instances are used for training.
- Non-sampled instances are used for validation.

Bootstrapping Cont'd

```
1 set.seed(1923)
2 booted <- bootstraps(train_df, times = 100)
3 booted
```

Bootstrap sampling

A tibble: 100 × 2

	splits	id
	<list>	<chr>
1	<split [379/140]>	Bootstrap001
2	<split [379/137]>	Bootstrap002
3	<split [379/138]>	Bootstrap003
4	<split [379/137]>	Bootstrap004
5	<split [379/136]>	Bootstrap005
6	<split [379/149]>	Bootstrap006
7	<split [379/142]>	Bootstrap007
8	<split [379/140]>	Bootstrap008
9	<split [379/141]>	Bootstrap009
10	<split [379/146]>	Bootstrap010

i 90 more rows

Doing Regularization in R

Data Split

```
1 library(mlbench)
2 library(tidymodels)
3 library(tidyverse)
4 tidymodels_prefer()
5 data("BostonHousing2")
6 df_new <- BostonHousing2 |>
7   mutate(target = log10(cmedv),
8           high_tax = ifelse(tax > 600, 1, 0)) |>
9   select(tract, target, nox, tax, high_tax)
10 set.seed(1292)
11 housing_split <- initial_split(df_new, prop = 0.75)
12 train_df <- training(housing_split)
13 test_df <- testing(housing_split)
```

Workflow

Recipe

Model

Workflow

```
1 # The order matters for this recipe
2 housing_recipe <- recipe(target ~ ., data = train_df) |>
3   update_role(tract, new_role = "id") |>
4   step_BoxCox(nox, tax) |>
5   step_range(all_numeric_predictors())
```

Workflow

Recipe

Model

Workflow

```
1 # Penalty = lambda; mixture = gamma
2 housing_model <- linear_reg(
3   penalty = tune(),
4   mixture = tune()
5 ) |>
6   set_mode("regression") |>
7   set_engine("glmnet")
```

Workflow

Recipe

Model

Workflow

```
1 housing_wf <- workflow() |>  
2   add_recipe(housing_recipe) |>  
3   add_model(housing_model)
```

Tuning the Model

Grid

CV

Iterating

Plot 1

Plot 2

Best

```
1 # Extract hyperparameter set and update space bounds if needed
2 glmnet_param <- extract_parameter_set_dials(housing_model)
3
4 # Create a space-filling design with 50 configurations
5 # For serious tuning, you'd probably increase this
6 set.seed(1561)
7 elastic_grid <- grid_space_filling(
8   glmnet_param,
9   size = 50,
10  type = "latin_hypercube"
11 )
12
13 print(elastic_grid, n = 6)
```

A tibble: 50 × 2

	penalty	mixture
	<dbl>	<dbl>
1	4.55e-10	0.506
2	1.57e- 3	0.247
3	2.01e- 5	0.600
4	1.95e-10	0.847
5	8.65e- 4	0.763
6	2.39e- 8	0.657

#> # A tibble: 44 × 2

Tuning the Model

Grid

CV

Iterating

Plot 1

Plot 2

Best

```
1 # 10-fold CV with 5 repeats
2 set.seed(1923)
3 elastic_cv <- vfold_cv(train_df, v = 10, repeats = 5)
```

Tuning the Model

Grid

CV

Iterating

Plot 1

Plot 2

Best

```
1 library(future)
2 library(doFuture)
3 # Setting evaluation metrics for validation
4 elastic_metrics <- metric_set(rsq)
5
6 # Setup modern, cross-platform parallel execution using all available cores
7 plan(multisession, workers = parallelly::availableCores())
8
9 # Register backend for foreach compatibility in tidymodels
10 registerDoFuture()
11
12 set.seed(1561)
13 elastic_tune <- housing_wf %>%
14   tune_grid(
15     resamples = elastic_cv,
16     grid = elastic_grid,
17     metrics = elastic_metrics,
18     control = control_grid(save_pred = TRUE, verbose = FALSE)
19   )
```

Tuning the Model

Grid

CV

Iterating

Plot 1

Plot 2

Best

```
1 # Plotting syntax: Mean R2 across folds by constellation
2 autoplot(elastic_tune) +
3   theme_light() +
4   labs(
5     title = "Hyperparameter Tuning for Elastic Net",
6     subtitle = "Space-Filling (Latin Hypercube) Design Results"
7   )
```

Tuning the Model

Grid

CV

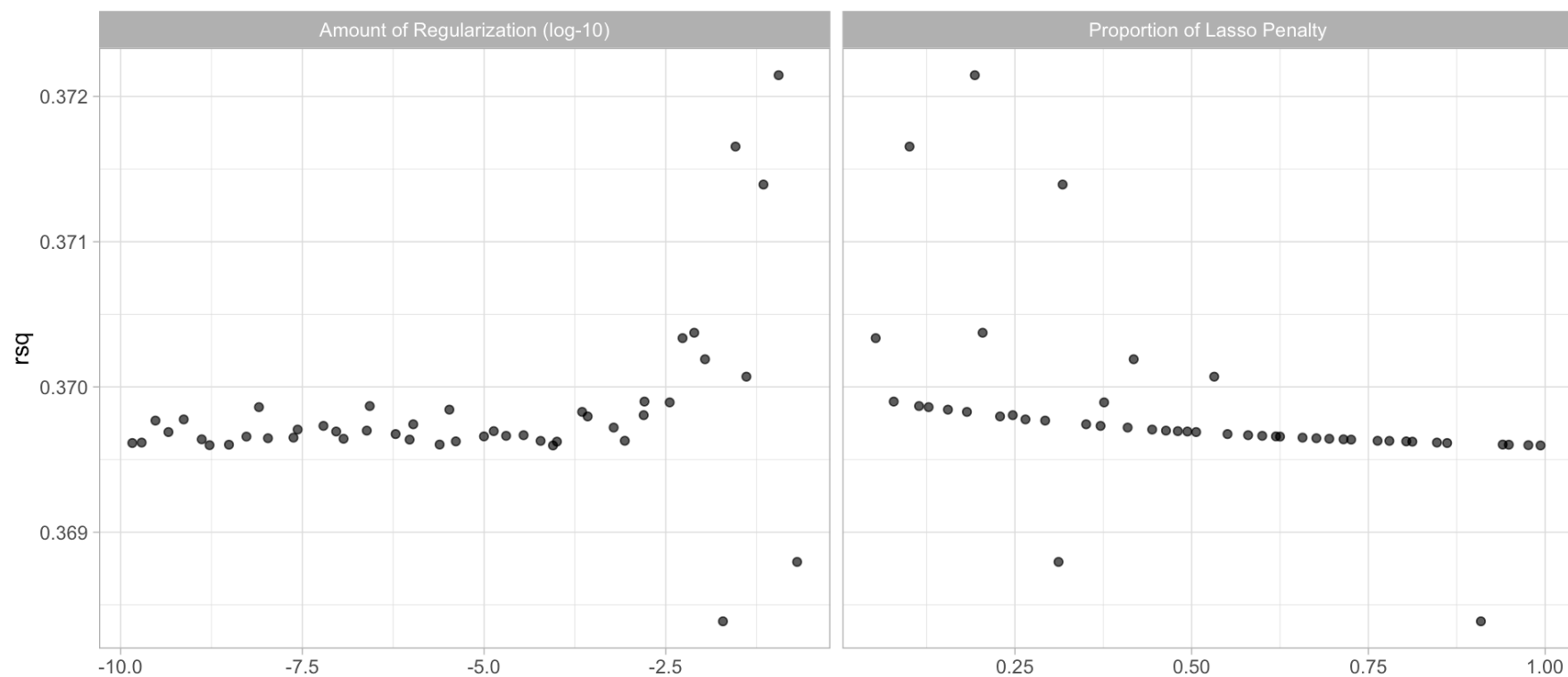
Iterating

Plot 1

Plot 2

Best

Hyperparameter Tuning for Elastic Net
Space-Filling (Latin Hypercube) Design Results



Tuning the Model

Grid

CV

Iterating

Plot 1

Plot 2

Best

```
1 # Extract best hyperparameter set
2 elastic_best <- select_best(elastic_tune, metric = "rsq") # Specify your target metric
3 elastic_best
```

A tibble: 1 × 3

penalty mixture .config

<dbl> <dbl> <chr>

```
1 0.113 0.193 pre0_mod46_post0
```

Final Fit

Syntax

Results

```
1 housing_updated <- finalize_model(housing_model,  
2                                   elastic_best)  
3 workflow_new <- housing_wf |>  
4   update_model(housing_updated)  
5 new_fit <- workflow_new |>  
6   fit(data = train_df)  
7 tidy_housing <- new_fit |>  
8   extract_fit_parsnip() |>  
9   tidy()
```

Final Fit

Syntax

Results

```
1 tidy_housing
```

```
# A tibble: 4 × 3
```

	term <chr>	estimate <dbl>	penalty <dbl>
1	(Intercept)	1.44	0.113
2	nox	-0.114	0.113
3	tax	-0.0879	0.113
4	high_tax	-0.0479	0.113

Performance

Syntax

Results

```
1 elastic_testfit <- new_fit |>
2   predict(test_df) |>
3   bind_cols(test_df) |>
4   metrics(truth = target, estimate = .pred)
```

Performance

Syntax

Results

```
1 elastic_testfit
```

```
# A tibble: 3 × 3
```

	.metric	.estimator	.estimate
	<chr>	<chr>	<dbl>
1	rmse	standard	0.145
2	rsq	standard	0.365
3	mae	standard	0.105

Exercise

Your Turn

Repeat the flow for a model containing all features, excepting “medv”, “town”, “lon”, and “lat”.

References

- Efron, Bradley. 1979. "Bootstrap Methods: Another Look at the Jackknife." *The Annals of Statistics* 7 (1): 1–26. <http://www.jstor.org/stable/2958830>.
- Efron, Bradley. 1983. "Estimating the Error Rate of a Prediction Rule: Improvement on Cross-Validation." *Journal of the American Statistical Association* 78 (382): 316–31. <https://doi.org/10.1080/01621459.1983.10477973>.
- Husslage, Bart G. M., Gijs Rennen, Edwin R. Van Dam, and Dick Den Hertog. 2011. "Space-Filling Latin Hypercube Designs for Computer Experiments." *Optimization and Engineering* 12 (4): 611–30. <https://doi.org/10.1007/s11081-010-9129-8>.
- McKay, M. D., R. J. Beckman, and W. J. Conover. 1979. "Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code." *Technometrics* 21 (2): 239–45. <https://doi.org/10.1080/00401706.1979.10489755>.
- Molinaro, Annette M., Richard Simon, and Ruth M. Pfeiffer. 2005. "Prediction Error Estimation: A Comparison of Resampling Methods." *Bioinformatics* 21 (15): 3301–7. <https://doi.org/10.1093/bioinformatics/bti499>.
- Tibshirani, Robert. 1996. "Regression Shrinkage and Selection Via the Lasso." *Journal of the Royal Statistical Society Series B: Statistical Methodology* 58 (1): 267–88. <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>.
- Tikhonov, Andrey. 1963. "Solution of Incorrectly Formulated Problems and the Regularization Method (О Решении Некорректно Поставленных Задач и Методе Регуляризации)." *Soviet Mathematics Doklady* 5: 1035–38.
- Yu, Qing Song, and Yi Zeng Liang. 2001. "Monte Carlo Cross Validation." *Chemometrics and Intelligent*